Engineering Cognition

Towards Persistent Understanding in AI Software Engineering

Mudit Sarda
30 June, 2026

1. Executive Summary

Software engineering is undergoing a fundamental transition. Large language models have rapidly evolved from code-completion tools into increasingly capable software engineering agents that can navigate repositories, modify large codebases, execute development tools, perform debugging tasks, and reason across complex software systems. As these capabilities continue to improve, code generation itself is becoming progressively less scarce. The limiting factor is no longer the ability to produce code, but the ability to accumulate engineering understanding over time.

Human software engineering is fundamentally cumulative. Every architectural investigation, debugging session, implementation attempt, design review, or production incident permanently changes how an experienced engineer understands a system. This continuously evolving internal representation influences future reasoning by capturing architectural constraints, implementation trade-offs, operational experience, subsystem behavior, historical design rationale, and the relationships between these concepts. Over time, this accumulated understanding compounds into a cognitive model that enables engineers to reason about increasingly complex software systems with progressively greater efficiency.

Contemporary AI coding agents possess very little of this cumulative understanding. Although they are increasingly capable of reasoning within the lifetime of a single interaction, much of the engineering understanding developed during that interaction disappears once the session ends. Future sessions frequently reconstruct architectural context, rediscover debugging conclusions, and repeat implementation investigations whose underlying engineering understanding has already been established. As a result, software engineering performed by today's AI agents remains largely episodic despite the inherently stateful nature of software development.

This paper argues that the missing abstraction is not memory in the conventional sense, but **Engineering Cognition**. We define **Engineering Cognition** as ”Engineering Cognition is the persistent understanding of a software system accumulated through engineering activity”. Unlike conversation history, code changes, or documentation, engineering cognition represents the persistent mental model that enables engineers to reason effectively about complex repositories

over extended periods of time. Conversations and implementations produce engineering cognition, but they are not themselves engineering cognition.

The central research question explored in this work is therefore not how AI agents should remember previous interactions, but whether engineering cognition itself admits a computational representation. If engineering understanding can be extracted, accumulated, organized, and reused across development sessions, it may become possible for AI software engineering systems to improve continuously rather than episodically. Such a capability would represent a shift from stateless code generation toward persistent engineering reasoning, potentially establishing engineering cognition as a new foundational layer of AI-native software development.

2. The Observation

The idea behind engineering cognition did not originate from a research hypothesis or an attempt to design a new software engineering system. It emerged from a relatively ordinary interruption during day-to-day development.

While working on a feature using OpenCode as my primary development environment, my terminal encountered a permissions issue that abruptly terminated the coding session. Because the implementation was already several prompts into development, the only practical way to continue was to open a new session and manually replay the previous interaction so that the agent could reconstruct enough context to resume the task. A short time later, the same interruption occurred again, requiring the entire process to be repeated.

Initially, this appeared to be little more than an inconvenience. After repeating the process several times, however, a more fundamental limitation became apparent. The coding agent had not merely forgotten the conversation—it had forgotten the engineering understanding that had emerged from the conversation. Architectural decisions, debugging discoveries, implementation rationale, and subsystem behaviour all had to be reconstructed despite having already been established during earlier work.

The obvious solution seemed to be preserving more history. Conversations could be archived, prompts stored, code changes indexed, and previous sessions retrieved whenever necessary. Existing coding systems already pursue many variations of this idea. The more this approach was considered, however, the less convincing it became. Preserving interactions does not necessarily preserve understanding. Conversations record how an engineer arrived at a conclusion, but not the engineering understanding that remains valuable after the conversation itself has ended.

This distinction becomes clear when considering experienced software engineers. Engineers rarely solve new problems by replaying months of previous discussions. Instead, they rely on an

internal understanding that has been continuously refined through implementation experience. They remember that a particular abstraction breaks under specific conditions, that an architectural constraint exists because several alternatives were previously explored, or that a subsystem behaves a certain way because of historical implementation decisions. The intermediate conversations that produced these conclusions are largely forgotten; the understanding remains.

This observation gradually shifted the focus away from engineering continuity itself. Session continuity was not the underlying problem. The real problem was the absence of persistent engineering understanding. Continuity simply emerged as a consequence of preserving that understanding over time.

The central question therefore, became substantially different from "How should AI coding agents remember previous conversations?" Instead, it became: **What is the computational representation of the engineering understanding that human engineers accumulate while building software?**

This paper refers to that representation as **Engineering Cognition**. Under this perspective, conversations, code changes, documentation, and debugging sessions are no longer treated as memory themselves. They are viewed as experiences from which engineering cognition emerges. Conversations are artefacts of reasoning; engineering cognition is the understanding that survives after the reasoning has ended.

3. Engineering Cognition as a Missing Layer

Software engineering is fundamentally a stateful activity. Every engineering decision is influenced not only by the current state of a codebase, but also by an accumulated understanding of how that system evolved over time. Architectural investigations, debugging sessions, implementation attempts, production incidents, and design discussions continuously refine an engineer's internal representation of the software system. Future engineering work is performed against this evolving cognitive model rather than against the repository alone.

This accumulated understanding explains much of the difference between novice and experienced engineers. Two engineers presented with the same repository and identical source code often reason very differently because one possesses years of accumulated engineering understanding while the other possesses only the observable artifacts of the system. The repository represents what the software is; engineering cognition represents how the engineer understands it.

Current AI coding agents possess only a limited analogue of this capability. They reason effectively within the context available to a single interaction, but that reasoning rarely persists beyond the lifetime of the session. Once the interaction concludes, much of the engineering

understanding developed during that reasoning process disappears. Subsequent sessions frequently reconstruct architectural context, rediscover implementation constraints, and repeat investigations that have already been performed. The underlying software evolves continuously while the agent's understanding repeatedly returns to an almost stateless baseline.

Existing approaches typically attempt to address this limitation by preserving additional information. Conversations are archived, documentation is indexed, repositories are embedded, and previous interactions become searchable. These approaches increase the amount of historical information available to the model, but they do not necessarily produce the continuously evolving engineering understanding that human engineers accumulate throughout software development. Information preservation and understanding accumulation are related, but they are not equivalent.

This paper argues that the missing abstraction is **Engineering Cognition**: the persistent, evolving representation of engineering understanding acquired through interaction with software systems. Engineering cognition is distinct from conversation history, source code, documentation, or implementation artifacts. It is the body of understanding that emerges from those artifacts and subsequently guides future engineering reasoning. In this view, conversations become evidence rather than memory, repositories become inputs rather than cognition, and software engineering becomes a process of continuously refining an evolving internal model of the system.

Treating engineering cognition as its own computational abstraction shifts the research question substantially. Rather than asking how AI agents should remember previous interactions, we instead ask whether engineering understanding itself can be represented, accumulated, retrieved, and refined computationally. If such a representation exists, persistent engineering cognition may become a fundamental capability for long-running AI software engineering systems rather than simply another mechanism for storing historical information.

Engineering cognition is not explicitly authored. It emerges through repeated interaction between engineers, software systems, and implementation activity.

4. Properties of Engineering Cognition

If engineering cognition is to be treated as a computational abstraction rather than simply another form of memory, it should satisfy a number of fundamental properties. These properties are not derived from any particular implementation. Instead, they arise from observing how experienced software engineers acquire, retain, and apply understanding throughout the lifetime of a software system.

First, engineering cognition should be **persistent**. Engineering understanding acquired during implementation, debugging, architectural investigation, or production operation should survive

beyond the individual interaction that produced it. Unlike conversational context, engineering cognition should evolve continuously throughout the lifetime of a repository.

Second, engineering cognition should be **hierarchical**. Not all engineering understanding possesses the same temporal significance. Temporary implementation discoveries, long-term architectural principles, and historical knowledge describing the evolution of a system each serve distinct purposes and naturally exist at different levels of permanence. A useful representation of engineering cognition should therefore preserve these distinctions rather than treating all knowledge uniformly.

Third, engineering cognition should be **reviewable**. Human engineers continuously refine their understanding by reinforcing correct conclusions while discarding incorrect assumptions. Any computational representation of engineering cognition should similarly permit understanding to be validated, revised, promoted, or rejected over time. Persistent cognition should emerge through continual refinement rather than indiscriminate accumulation.

Fourth, engineering cognition should be **relevant**. As engineering understanding compounds, only a small fraction of accumulated cognition is likely to influence any particular engineering task. Effective cognition therefore depends not only on accumulation, but also on the ability to retrieve the subset of understanding most applicable to the current reasoning problem.

Fifth, engineering cognition should be **portable**. Engineering understanding belongs to the engineer's interaction with the software system rather than to any individual language model, coding agent, or development environment. A representation of engineering cognition should therefore remain usable across changing models, tools, and software engineering workflows.

Finally, engineering cognition should be **compounding**. Every implementation task, debugging investigation, architectural review, or production incident should incrementally improve the quality of future engineering reasoning. The value of engineering cognition therefore lies not in preserving historical information, but in enabling understanding to become progressively richer through continued interaction with the software system.

Collectively, these properties distinguish engineering cognition from existing notions of conversational memory or historical information retrieval. Rather than representing what previously occurred, engineering cognition represents the evolving body of understanding that enables increasingly effective software engineering over time.

5. Why Existing Approaches Are Not Engineering Cognition

Recent AI coding systems have introduced increasingly sophisticated mechanisms for preserving information across software engineering sessions. Although these approaches differ

considerably in implementation, they largely share the same underlying objective: to make previously observed information available for future retrieval. This represents a meaningful improvement over completely stateless interactions, but information preservation should not be conflated with the accumulation of engineering understanding.

One common approach is **conversation persistence**, where previous interactions remain available for future sessions. This improves continuity by allowing models to revisit earlier discussions without completely reconstructing context. However, conversations primarily record the process of reasoning rather than the engineering understanding produced by that reasoning. Exploratory discussions, unsuccessful implementation attempts, intermediate hypotheses, and debugging dead ends all become permanent despite contributing only indirectly to future engineering decisions.

A second class of systems attempts to address this limitation through **conversation summarization**. Compressing long interactions into progressively shorter summaries improves retrieval efficiency while reducing storage requirements. Nevertheless, summaries remain compressed descriptions of previous interactions rather than explicit representations of engineering understanding. Repeated summarization further compounds the problem by gradually discarding contextual information as summaries themselves become inputs to future summaries.

Many modern coding agents instead rely on **semantic retrieval**, retrieving conversations, documentation, design notes, or previous implementations that appear relevant to the current task. This substantially improves discoverability across large bodies of historical information. However, semantic similarity answers a fundamentally different question from engineering cognition. It identifies information that resembles the current problem rather than understanding that should influence the current engineering decision.

Repository-scale **code indexing** extends this idea further by constructing searchable representations of entire codebases. These systems enable agents to efficiently navigate large repositories, identify relevant files, and retrieve implementation context beyond the immediate prompt. While this significantly improves repository awareness, indexing remains fundamentally concerned with representing software artifacts rather than representing the understanding engineers develop while interacting with those artifacts. Knowing where information exists is distinct from understanding what has been learned from it.

The distinction becomes clearer when considering experienced software engineers. Engineers rarely solve new problems by searching every previous conversation, rereading every design document, or revisiting every historical implementation. Instead, they rely on an internal understanding that has been continuously refined through experience. Conversations, documentation, code, and debugging sessions contribute to this understanding, but they are not themselves the understanding.

Existing approaches therefore preserve **information**. Engineering cognition attempts to preserve **understanding**. Information records what previously occurred; engineering cognition

represents the evolving body of engineering understanding that emerges from those experiences and subsequently guides future reasoning. If engineering cognition is indeed a meaningful computational abstraction, it cannot be reduced to conversation history, document retrieval, semantic search, or repository indexing alone.

6. Operationalizing Engineering Cognition

If engineering cognition is accepted as a meaningful computational abstraction, the next question becomes how such an abstraction might be represented within an AI software engineering system.

The central objective is no longer preserving conversations or retrieving historical information, but constructing a representation that continuously accumulates engineering understanding while remaining useful during future reasoning. Multiple computational realizations of such a system are conceivable, each making different trade-offs in representation, retrieval, review, and evolution.

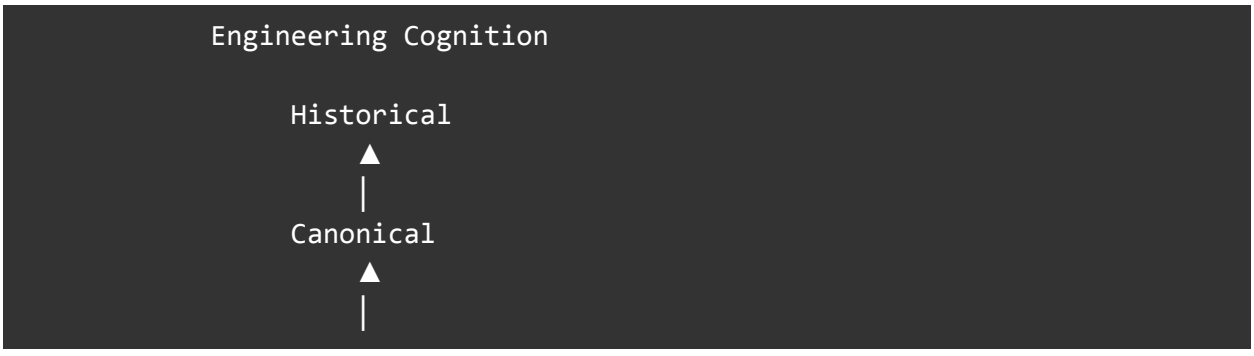
The architecture presented in this paper should therefore be viewed as **one experimental operationalization** of engineering cognition rather than its definitive implementation. The intent is not to claim that engineering cognition admits a unique representation, but to demonstrate that the abstraction can be implemented as a practical engineering system whose behavior approximates the way experienced software engineers accumulate understanding over time.

Engineering Memory System (EMS) is the implementation currently being developed to explore this hypothesis.

6.1 Cognition Hierarchy

Engineering understanding does not emerge fully formed. Observations produced during active software development gradually evolve into durable engineering knowledge, while the long-term evolution of a repository provides historical context that explains why the system exists in its current form.

EMS represents this progression through a hierarchical cognition model.



```
                    Session
```

Session Cognition represents understanding generated while solving an active engineering task. It captures temporary implementation discoveries, debugging observations, architectural hypotheses, and other forms of working cognition that exist primarily to support the current engineering session.

Canonical Cognition contains reviewed engineering understanding that has been accepted as durable knowledge about the software system. This layer represents the persistent engineering model inherited by future development sessions and continuously evolves as additional understanding is validated.

Historical Cognition captures understanding derived from the evolution of the repository itself. Rather than describing individual code changes, it attempts to preserve the engineering rationale that explains how architectural decisions emerged over time, providing long-term contextual understanding beyond the current implementation.

The hierarchy intentionally separates transient reasoning from persistent engineering understanding, allowing cognition to mature naturally as software systems evolve.

6.2 Cognition Lifecycle

Engineering cognition is not created instantaneously. It emerges gradually through repeated interaction with a software system, where individual engineering activities produce observations that may eventually become durable understanding. Rather than treating cognition as static documentation, EMS models it as a continuously evolving process whose output is refined over time through human judgment.

The proposed lifecycle is illustrated below.

```
Engineering Session
        │
        ▼
Engineering Observations
        │
        ▼
Understanding Extraction
        │
        ▼
Candidate Cognition
        │
```

Each engineering session generates numerous observations, including implementation discoveries, debugging outcomes, architectural insights, and inferred system behavior. Rather than preserving these observations directly, EMS attempts to extract higher-level engineering understanding from them. The extracted understanding is represented as candidate cognition, which remains provisional until reviewed by the engineer responsible for the work. For instance, if this is the LLM observation:
“TokenManager refreshes the JWT before invalidating the cache.”
then a weak cognition will be “Authentication retries before cache invalidation” and a stronger cognition shall be something like “Authentication correctness depends on optimistic token refresh. Future authentication implementations should preserve this invariant.”

Only reviewed cognition is promoted into canonical cognition, ensuring that persistent engineering understanding evolves under developer supervision rather than through automatic accumulation. During subsequent software development, relevant canonical cognition is retrieved to influence future reasoning, allowing engineering understanding to compound across successive engineering sessions.

This lifecycle represents one possible operationalization of persistent engineering cognition. Alternative systems may choose different extraction mechanisms, review processes, or promotion criteria while preserving the same underlying objective: enabling engineering understanding to accumulate over time rather than being repeatedly reconstructed.

6.3 Cognition Retrieval

Persistent engineering cognition is valuable only if the relevant understanding can participate in future reasoning. Simply accumulating engineering cognition is therefore insufficient; the system must also determine which subset of accumulated understanding should influence the current engineering task.

EMS approaches this through a retrieval pipeline that reconstructs task-specific engineering context dynamically rather than replaying historical interactions.

The proposed retrieval process is illustrated below.

The process begins by analyzing the engineering intent expressed by the developer's prompt. Rather than retrieving all previously accumulated cognition, the system identifies engineering understanding that is most relevant to the current task. Retrieved cognition is then ranked according to its expected contribution to the reasoning process before being incorporated into the prompt supplied to the language model.

The objective is not to maximize the amount of retrieved information, but to maximize the quality of engineering reasoning. As accumulated cognition grows throughout the lifetime of a repository, increasingly selective retrieval becomes essential for maintaining coherent and contextually relevant reasoning within bounded inference contexts.

This perspective distinguishes engineering cognition from conventional retrieval systems. Traditional retrieval attempts to recover previously stored information. Cognition retrieval instead attempts to reconstruct the engineering understanding most likely to improve the reasoning process itself.

6.4 Human-Governed Cognition

Unlike conventional memory systems, engineering cognition cannot be accumulated automatically.

Language models are capable of generating plausible engineering conclusions from implementation activity. However, plausibility is not equivalent to engineering understanding.

Intermediate reasoning may contain incorrect assumptions, incomplete abstractions, or observations that appear locally correct but fail to generalize across the broader software system. If every generated conclusion were promoted into persistent cognition, the quality of accumulated understanding would progressively deteriorate.

EMS therefore treats engineering cognition as a human-governed artifact rather than an automatically generated one.

During each engineering session, the system extracts candidate engineering cognition from the development process. These candidates remain provisional until explicitly reviewed by the engineer responsible for the work. Only reviewed cognition is allowed to influence future engineering sessions.

This review process serves a purpose beyond simple error correction. Human engineers continuously refine their own mental models, discarding outdated assumptions while strengthening abstractions that prove consistently useful across multiple engineering tasks. Engineering cognition should evolve through the same process. Review therefore becomes part of cognition formation rather than merely a validation step.

This fundamentally distinguishes engineering cognition from conventional memory systems. Traditional memory attempts to preserve everything that occurred. Engineering cognition instead preserves only understanding that has survived engineering judgment. The objective is not to maximize stored information, but to maximize the quality of accumulated understanding available to future reasoning.

7. Open Research Questions

Treating engineering cognition as a first-class computational abstraction raises a number of research questions that remain largely unexplored. Although this paper proposes one possible realization through Engineering Memory System (EMS), the broader scientific problem extends well beyond any individual implementation.

Perhaps the most fundamental question is **what engineering cognition actually is**. Human software engineers continuously construct, refine, merge, and reinterpret their understanding of a software system as they work. This understanding is clearly more than conversation history, code changes, or documentation, yet it is not obvious how it should be formally represented. Developing a rigorous definition of engineering cognition is therefore a prerequisite for any persistent cognition system.

Closely related is the question of **whether engineering cognition can be extracted automatically**. Contemporary language models readily summarize conversations and identify implementation facts, but engineering understanding is significantly more abstract. It remains

unclear whether models can reliably distinguish durable engineering insight from transient implementation details, or whether new extraction methodologies are required.

Even if engineering cognition can be extracted, another question immediately follows: **does it admit a canonical representation?** Human engineers rarely preserve their understanding as static documents. Instead, their internal models evolve continuously as new evidence appears. Whether a persistent engineering cognition system should similarly support evolving representations, rather than immutable knowledge objects, remains an open problem.

Evaluation presents another challenge. Existing benchmarks primarily measure code generation, task completion, or repository navigation. They provide little insight into whether an agent has accumulated engineering understanding over extended periods of development. Developing rigorous methodologies for measuring engineering cognition therefore represents an important research direction.

Ultimately, the central empirical question is whether **persistent engineering cognition measurably improves long-horizon software engineering**. If engineering understanding can be accumulated across sessions, repositories, and development cycles, does this lead to better architectural reasoning, fewer repeated investigations, faster debugging, or more consistent implementation decisions? Answering this question requires carefully designed longitudinal experiments rather than anecdotal evidence.

Beyond individual repositories, additional questions naturally emerge. Can engineering cognition transfer across unrelated software systems in the same way experienced engineers transfer intuition between projects? Can multiple engineers collaboratively construct shared engineering cognition without degrading its quality? How should conflicting engineering understanding be reconciled? These questions suggest that persistent engineering cognition may eventually extend beyond individual developers toward collective engineering intelligence.

Collectively, these questions define a broader research program whose objective is not merely to build better memory systems for coding agents, but to understand how engineering understanding itself can be represented, accumulated, evaluated, and reused by AI systems over time.

8. Future Directions

Engineering cognition is unlikely to become a mature computational abstraction through a single implementation. Instead, this work should be viewed as the starting point of a broader research program whose objective is to understand how engineering understanding can be represented, accumulated, and reused by AI systems over long time horizons.

A natural first direction is the study of **engineering cognition extraction**. Current language models readily identify implementation facts and summarize engineering activity, but distinguishing durable engineering understanding from transient observations remains largely

unsolved. Developing principled extraction methodologies may ultimately prove to be the central challenge of persistent engineering cognition.

A second direction concerns **evaluation**. Existing benchmarks primarily measure code generation, repository navigation, or task completion, providing little insight into whether an agent has accumulated engineering understanding. New evaluation methodologies will likely be required to quantify engineering cognition itself and measure how it evolves over extended software development.

Closely related are **long-horizon agent performance experiments**. An important empirical question is whether persistent engineering cognition produces measurable improvements in software engineering tasks such as architectural reasoning, debugging efficiency, implementation consistency, or recovery from interrupted development. Answering this question will require controlled experiments spanning multiple sessions, repositories, and engineering workflows.

Beyond individual developers lies the question of **collaborative engineering cognition**. Human engineering teams collectively construct shared understanding despite each engineer possessing a unique mental model of the system. Whether engineering cognition can be shared, merged, or transferred between engineers while preserving quality remains an open area of investigation.

Finally, if engineering cognition proves to be a useful abstraction, it may motivate an entirely new class of systems: **Cognition Operating Systems**. Rather than functioning solely as memory layers for individual coding agents, such systems would manage the complete lifecycle of engineering cognition, including its extraction, organization, refinement, retrieval, evolution, and transfer across software systems, development environments, and AI agents.

The long-term objective is therefore broader than improving today's coding assistants. It is to establish engineering cognition as a computational primitive for AI-native software engineering and to understand how persistent engineering understanding can become an enduring component of future intelligent software development systems.

9. Conclusion

Software engineering has always been a cumulative discipline. Engineers become more effective not simply because they reason well, but because every implementation, debugging investigation, architectural review, and production incident permanently refines their understanding of the systems they build. Over time, this accumulated understanding becomes a continuously evolving mental model that guides future engineering decisions.

Contemporary AI software engineering remains fundamentally episodic. Despite remarkable progress in code generation, repository navigation, and tool use, most AI coding agents begin each new engineering session with little of the understanding developed during previous work. Conversations may persist, repositories may be indexed, and historical information may be retrievable, yet the engineering understanding itself is rarely accumulated in a form that continuously improves future reasoning.

This paper argues that the missing abstraction is **engineering cognition**: the evolving body of engineering understanding that emerges through sustained interaction with software systems. We further present Engineering Memory System (EMS) as one experimental operationalization of this abstraction, not as its definitive implementation, but as evidence that persistent engineering cognition can be explored as a practical systems problem.

Whether engineering cognition admits a computational representation remains an open scientific question. It is not yet known how engineering understanding should be extracted, represented, evaluated, transferred, or evolved over long periods of software development. These questions extend well beyond any individual implementation and define a broader research agenda at the intersection of software engineering, artificial intelligence, and long-horizon autonomous agents.

If software engineering continues its transition toward increasingly capable AI collaborators, the ability to accumulate engineering understanding may become as important as the ability to generate code itself. Engineering cognition therefore represents not only an opportunity to improve today's coding agents, but also a possible foundation for the next generation of AI-native software engineering.

We believe that understanding how engineering cognition can be represented computationally is a worthwhile research direction—one whose answers may ultimately determine how AI systems learn to become not only capable programmers, but progressively better software engineers.